

NETWORKING — COMPLETE CHEAT SHEET

VIMAAN L2 Support Engineer Interview Prep

TABLE OF CONTENTS

1. Core Networking Concepts (OSI/TCP-IP)
 2. IP Addressing & Subnetting
 3. DNS — Deep Dive
 4. DHCP — Deep Dive
 5. TCP vs UDP
 6. SSH & Remote Access
 7. VPN Concepts
 8. Firewalls (iptables/firewalld/ufw)
 9. Network Diagnostic Commands
 10. Packet Capture with tcpdump
 11. Common Ports Reference
 12. LDAP & Authentication Services
 13. Troubleshooting Scenarios
 14. Interview Q&A Bank
-

1. CORE NETWORKING CONCEPTS

1.1 OSI Model (7 Layers) — Classic interview question

Layer	Name	Examples
7	Application	HTTP, DNS, SSH, FTP
6	Presentation	Encryption, SSL/TLS, encoding
5	Session	Session establishment/teardown
4	Transport	TCP, UDP
3	Network	IP, routing, ICMP
2	Data Link	MAC addresses, switches, ARP
1	Physical	Cables, NICs, electrical signals

Mnemonic: "All People Seem To Need Data Processing" (7→1)

1.2 TCP/IP Model (4 Layers — what's actually used in practice)

```
Application  (HTTP, DNS, SSH — combines OSI 5-7)
Transport    (TCP, UDP — OSI 4)
Internet     (IP, ICMP — OSI 3)
Link         (Ethernet, MAC — OSI 1-2)
```

1.3 "What happens when you type a URL into a browser?" — Very common opener

1. Browser checks local DNS cache, then OS cache

2. If not cached, DNS resolver queries → root servers → TLD servers → authoritative nameserver → returns IP
 3. Browser opens TCP connection to that IP (3-way handshake: SYN → SYN-ACK → ACK)
 4. If HTTPS: TLS handshake happens (certificate exchange, key negotiation)
 5. Browser sends HTTP GET request
 6. Server processes request, sends back HTTP response
 7. Browser renders the HTML/CSS/JS content
 8. Connection closes (or stays alive if keep-alive is used)
-

2. IP ADDRESSING & SUBNETTING

2.1 IPv4 Basics

- 32-bit address, written as 4 octets: `192.168.1.10`
- Each octet: 0-255
- **Private IP ranges (won't route on the public internet):**
 - `10.0.0.0 – 10.255.255.255` (10.0.0.0/8)
 - `172.16.0.0 – 172.31.255.255` (172.16.0.0/12)
 - `192.168.0.0 – 192.168.255.255` (192.168.0.0/16)
- **Loopback:** `127.0.0.1` (localhost)

2.2 Subnet Masks & CIDR

CIDR	Subnet Mask	Usable Hosts
/8	255.0.0.0	~16.7 million

CIDR	Subnet Mask	Usable Hosts
/16	255.255.0.0	~65,534
/24	255.255.255.0	254
/25	255.255.255.128	126
/28	255.255.255.240	14
/30	255.255.255.252	2 (point-to-point links)
/32	255.255.255.255	1 (single host)

Quick subnetting formula: Usable hosts = $2^{(32-\text{CIDR})} - 2$ (subtract network + broadcast address)

Example: `192.168.1.0/24` → network address `192.168.1.0`, broadcast `192.168.1.255`, usable range `192.168.1.1 – 192.168.1.254`

2.3 IPv6 (brief but be aware)

- 128-bit address, written in hex, colon-separated: `2001:0db8::1`
- No more NAT-dependency, much larger address space
- `::1` is loopback (like 127.0.0.1 in IPv4)

3. DNS — DEEP DIVE

3.1 DNS Resolution Flow

User types google.com

↓
Check browser cache → OS cache (/etc/hosts, resolver cache)
↓ (if not found)
Query configured DNS resolver (e.g., ISP DNS or 8.8.8.8)
↓
Resolver queries Root Name Server → gets referral to .com TLD server
↓
Resolver queries TLD Name Server → gets referral to authoritative NS for google.com
↓
Resolver queries Authoritative Name Server → gets the actual IP
↓
Resolver caches result (per TTL) and returns IP to client

3.2 DNS Record Types

Record	Purpose
A	Maps hostname → IPv4 address
AAAA	Maps hostname → IPv6 address
CNAME	Alias — maps hostname → another hostname
MX	Mail exchange server for a domain
NS	Nameserver responsible for the domain
PTR	Reverse lookup — IP → hostname
TXT	Arbitrary text (SPF, domain verification, etc.)
SOA	Start of Authority — zone's primary info

3.3 DNS Commands

```
bash

nslookup google.com                # Basic lookup
nslookup google.com 8.8.8.8        # Query specific DNS server
dig google.com                     # Detailed DNS query tool
dig google.com +short              # Just the IP, no verbose output
dig google.com MX                  # Query specific record type
dig -x 8.8.8.8                     # Reverse DNS lookup
dig google.com @8.8.8.8            # Query using specific nameserver
host google.com                    # Simple lookup alternative
cat /etc/resolv.conf               # Configured DNS servers on this machine
cat /etc/hosts                     # Local hostname overrides (checked before)
systemd-resolve --status           # DNS resolver status (modern Ubuntu)
```

3.4 Common DNS Issues & Fixes

- **Symptom: "Can ping IP but not hostname"** → DNS resolution issue, check `/etc/resolv.conf`, try `dig`/`nslookup`
 - **Symptom: Stale DNS entries** → check TTL, flush local cache: `systemd-resolve --flush-caches`
 - **Symptom: Intermittent resolution failures** → check if resolver is reachable, try alternate DNS (8.8.8.8, 1.1.1.1)
-

4. DHCP — DEEP DIVE

4.1 DHCP Process — "DORA" (Common interview question)

1. DISCOVER — Client broadcasts to find DHCP servers (no IP yet)
2. OFFER — Server responds with an available IP lease offer
3. REQUEST — Client broadcasts, requesting to accept that offer (in case multiple servers responded)
4. ACK — Server confirms lease, client configures its interface

4.2 DHCP Lease Details

- Lease has a duration (e.g., 24 hours); client tries to renew at 50% of lease time
- If server unreachable, client tries again at 87.5% of lease time
- If lease expires entirely, client goes back to DISCOVER

4.3 Commands

```
bash

dhclient eth0          # Request/renew DHCP lease manually
dhclient -r eth0       # Release current lease
ip addr show           # Check current IP assignment
cat /var/lib/dhcp/dhclient.leases # View lease info (Ubuntu)
journalctl -u NetworkManager # DHCP-related logs (systems using NetworkManager)
```

DHCP server config file (if you're running one): `/etc/dhcp/dhcpd.conf`

5. TCP vs UDP

	TCP	UDP
Connection	Connection-oriented (handshake)	Connectionless
Reliability	Guaranteed delivery, retransmits	No guarantee, "fire and forget"
Ordering	Ordered delivery	No ordering guarantee
Speed	Slower (overhead of reliability)	Faster, lower overhead
Use cases	HTTP/HTTPS, SSH, FTP, email	DNS queries, streaming, VoIP, gaming

TCP 3-Way Handshake:

```
Client → SYN → Server
Client ← SYN-ACK ← Server
Client → ACK → Server
(connection established)
```

TCP 4-Way Termination:

```
Client → FIN → Server
Client ← ACK ← Server
Client ← FIN ← Server
Client → ACK → Server
```

6. SSH & REMOTE ACCESS

bash

```
ssh user@host # Basic connection
ssh -p 2222 user@host # Custom port
ssh -i ~/.ssh/id_rsa user@host # Specify key
ssh -v user@host # Verbose (troubleshooting)
ssh -vvv user@host # Extra verbose
ssh -L 8080:localhost:80 user@host # Local port forwarding
ssh -R 8080:localhost:80 user@host # Remote port forwarding
ssh -D 1080 user@host # Dynamic port forwarding (SOCKS proxy)
ssh -N -f -L ... # Run tunnel in background, no shell
scp file.txt user@host:/path/ # Copy file to remote
scp -r dir/ user@host:/path/ # Copy directory
sftp user@host # Interactive file transfer session
```

Key config files:

- `/etc/ssh/sshd_config` — server-side config
- `~/.ssh/config` — client-side per-host config (aliases, keys, ports)
- `~/.ssh/authorized_keys` — public keys allowed to authenticate
- `~/.ssh/known_hosts` — fingerprints of previously connected hosts

Common sshd_config hardening settings:

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
Port 2222
MaxAuthTries 3
AllowUsers john jane
```

7. VPN CONCEPTS

- **Purpose:** Creates an encrypted tunnel over an untrusted network (internet) so remote systems can securely communicate as if on the same private network.
- **Common types:**
 - **Site-to-Site VPN** — connects two networks/offices permanently
 - **Remote Access VPN** — individual users connect to a corporate network
- **Common protocols:** IPSec, OpenVPN, WireGuard, L2TP
- **Key concepts:** tunneling, encryption (AES), authentication (pre-shared keys or certificates), split-tunneling vs full-tunneling

```
bash
ip a show tun0           # Check VPN tunnel interface (common name)
systemctl status openvpn # Check OpenVPN service status
wg show                  # WireGuard status (if using WireGuard)
```

8. FIREWALLS

8.1 iptables (traditional, still widely used/understood)

```
bash
```

```
iptables -L -n -v                # List all rules, verbose, numeric
iptables -L -n --line-numbers     # List with line numbers (for deletion)
iptables -A INPUT -p tcp --dport 22 -j ACCEPT  # Allow SSH inbound
iptables -A INPUT -p tcp --dport 80 -j ACCEPT  # Allow HTTP inbound
iptables -A INPUT -j DROP         # Drop everything else (careful!)
iptables -D INPUT 3               # Delete rule number 3
iptables-save > /etc/iptables/rules.v4        # Persist rules (Ubuntu)
iptables -F                       # Flush all rules (careful!)
```

8.2 firewalld (RHEL/CentOS default)

```
bash

firewall-cmd --state              # Check if running
firewall-cmd --list-all          # Show current zone's rules
firewall-cmd --get-active-zones   # Show active zones
firewall-cmd --add-service=http --permanent  # Allow HTTP permanently
firewall-cmd --add-port=8080/tcp --permanent # Allow specific port
firewall-cmd --reload             # Apply permanent changes
firewall-cmd --remove-service=http --permanent # Remove a rule
firewall-cmd --list-ports         # List open ports
```

8.3 ufw (Ubuntu Uncomplicated Firewall)

```
bash
```

```
ufw status verbose           # Check status and rules
ufw enable / ufw disable    # Toggle firewall
ufw allow 22/tcp             # Allow SSH
ufw allow from 192.168.1.0/24 # Allow entire subnet
ufw deny 23                  # Deny telnet
ufw delete allow 22          # Remove a rule
ufw default deny incoming   # Set default policy
```

8.4 IDS Concept (Intrusion Detection System)

- Monitors network/system for malicious activity or policy violations, generates alerts (doesn't block, unlike IPS which can actively block)
 - Examples: Snort, Suricata, OSSEC
 - **Firewall vs IDS:** firewall enforces rules to allow/block traffic; IDS passively monitors and alerts on suspicious patterns
-

9. NETWORK DIAGNOSTIC COMMANDS

```
bash
```

```
ip a                # Show all interfaces & IPs (modern)
ifconfig            # Show interfaces (legacy, still common)
ip link show        # Show interface status (up/down)
ip route            # Show routing table
route -n            # Routing table (legacy)
ping -c 4 google.com # Send 4 ICMP echo requests
ping -i 0.5 host     # Custom interval
traceroute google.com # Trace path/hops to destination
mtr google.com       # Combines ping + traceroute, live updating
netstat -tulnp       # Listening ports + PID (legacy, may need i
ss -tulnp           # Listening ports + PID (modern replacement)
ss -s               # Socket statistics summary
curl -I https://example.com # HTTP headers only, quick status check
curl -v https://example.com # Verbose – see full request/response + TLS
curl -o /dev/null -s -w "%{http_code}\n" https://example.com # Just the sta
wget https://example.com/file # Download a file
telnet host 80       # Test raw TCP connectivity to a port
nc -zv host 80       # netcat – quick port scan/check
nmap -p 1-1000 host  # Port scan a range (if nmap available)
arp -a              # Show ARP cache (IP-to-MAC mappings)
ip neigh            # Modern equivalent of arp -a
```

10. PACKET CAPTURE WITH TCPDUMP

```
bash
```

```

tcpdump -i eth0                                # Capture on interface eth0
tcpdump -i eth0 port 80                        # Filter by port
tcpdump -i eth0 host 192.168.1.10             # Filter by host
tcpdump -i eth0 src 192.168.1.10              # Filter by source IP
tcpdump -i eth0 dst 192.168.1.20              # Filter by destination IP
tcpdump -i eth0 -w capture.pcap                # Write to file for later analysis
tcpdump -r capture.pcap                       # Read from saved file
tcpdump -i eth0 -c 100                        # Capture only 100 packets
tcpdump -i eth0 icmp                          # Capture only ICMP (ping) traffic
tcpdump -i eth0 -A port 80                    # Show packet content in ASCII (hex)
tcpdump -i any -nn port 443                   # -nn avoids DNS/service name resolution

```

When to reach for tcpdump in L2 support: connectivity issues that ping/curl can't explain, verifying if traffic is actually leaving/arriving at the server, confirming firewall is dropping vs application not responding.

11. COMMON PORTS REFERENCE (Memorize these — guaranteed to come up)

Port	Protocol	Service
20/21	TCP	FTP (data/control)
22	TCP	SSH / SFTP
23	TCP	Telnet (insecure, legacy)
25	TCP	SMTP (mail sending)
53	TCP/UDP	DNS

Port	Protocol	Service
67/68	UDP	DHCP (server/client)
80	TCP	HTTP
110	TCP	POP3
123	UDP	NTP
143	TCP	IMAP
161/162	UDP	SNMP
389	TCP	LDAP
443	TCP	HTTPS
445	TCP	SMB (Windows file sharing)
514	UDP	Syslog
636	TCP	LDAPS (LDAP over SSL)
993	TCP	IMAPS
995	TCP	POP3S
2049	TCP/UDP	NFS
3306	TCP	MySQL
3389	TCP	RDP (Windows Remote Desktop)
5432	TCP	PostgreSQL

Port	Protocol	Service
6379	TCP	Redis
8080	TCP	HTTP alternate (common for apps/proxies)
9090	TCP	Prometheus (default)
9100	TCP	Node Exporter (Prometheus)
3000	TCP	Grafana (default)

12. LDAP & AUTHENTICATION SERVICES

- **LDAP (Lightweight Directory Access Protocol)** — used for centralized authentication/directory services (like a phonebook for users/groups/permissions). Active Directory is Microsoft's implementation built on LDAP.
- **Common use case in L2 support:** Linux servers joined to an LDAP/AD domain for centralized user login instead of local `/etc/passwd` accounts.

```
bash

ldapsearch -x -H ldap://ldapserver -b "dc=example,dc=com" # Basic LDAP query
getent passwd username # Check if LDAP entry exists
id username # Confirm UID/GID
```

Key files: `/etc/nsswitch.conf` (defines lookup order — files, ldap, sss),
`/etc/sss/sss.conf` (SSSD — System Security Services Daemon, commonly used to integrate Linux with AD/LDAP).

13. TROUBLESHOOTING SCENARIOS

Scenario: "Customer says they can't reach our server"

```
bash

ping <server-ip>                # Basic reachability
traceroute <server-ip>          # Where does it break?
telnet <server-ip> <port>        # Is the specific port open?
curl -v https://server          # Full request/response detail
# Check server side:
ss -tulnp | grep <port>         # Is the service actually listening?
systemctl status <service>      # Is the service running?
firewall-cmd --list-all / ufw status # Is firewall blocking it?
tcpdump -i eth0 port <port>     # Is traffic even arriving?
```

Scenario: "Intermittent connectivity / packet loss"

```
bash

mtr <host>                      # Live hop-by-hop loss %
ping -c 100 <host> | tail -3    # Statistical packet loss over 100 pings
```

Scenario: "DNS not resolving"

```
bash

cat /etc/resolv.conf            # Check configured DNS servers
dig <domain> @8.8.8.8           # Test against known-good DNS server
nslookup <domain>               # Compare against local resolver
```

Scenario: "High latency to a service"

```
bash
```

```
mtr <host>
```

```
# Identify which hop introduces latency
```

```
ping <host>
```

```
# Baseline round-trip time
```

14. INTERVIEW Q&A BANK

Q: Explain the TCP 3-way handshake. A: SYN (client requests connection) → SYN-ACK (server acknowledges and requests back) → ACK (client confirms) — connection is now established and both sides can send data reliably.

Q: Difference between TCP and UDP, with examples? A: TCP is connection-oriented and reliable (retransmits lost packets, guarantees order) — used for HTTP, SSH, email. UDP is connectionless and best-effort, no delivery guarantee, but faster/lower overhead — used for DNS queries, video streaming, VoIP where speed matters more than occasional loss.

Q: What is DORA in DHCP? A: Discover (client broadcasts for a DHCP server), Offer (server offers an IP), Request (client requests that specific offer), Acknowledge (server confirms and finalizes the lease).

Q: How would you troubleshoot "server unreachable"? A: Start with `ping` to check basic reachability, `tracert`/`tracroute`/`mtr` to see where the path breaks, then `telnet`/`nc` to check if the specific port is open. If ping works but the port doesn't respond, it's likely a firewall or the service isn't running — check with `ss -tulnp` and `systemctl status` on the server.

Q: What's the difference between a firewall and an IDS? A: A firewall actively allows or blocks traffic based on rules (ports, IPs, protocols). An IDS passively monitors traffic/system activity and generates alerts on suspicious patterns but doesn't block traffic itself (an IPS does actively block).

Q: What is NAT and why is it used? A: Network Address Translation maps private internal IP addresses to a public IP for internet communication, allowing many internal devices to share one public IP — also adds a layer of obscurity/security since internal addressing isn't directly exposed.

Q: How do you check what's listening on a port? A: `ss -tulnp | grep <port>` (or `netstat -tulnp` on older systems) shows the process and PID bound to that port.